



Unidade 2 — Frameworks



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Identificar** os principais frameworks Back-End e Front-End;
- **Comparar** frameworks de diferentes linguagens e paradigmas;
- **Explicar** as características de cada framework;
- **Avaliar** qual framework é mais adequado para cada cenário.

Taxonomia de Bloom: Identificar, Comparar, Explicar, Avaliar.



Frameworks Back-End

Framework	Linguagem	Características	Ideal para
Express	JavaScript	Minimalista, flexível, grande ecossistema	APIs, SPAs, microserviços
NestJS	TypeScript	Arquitetura modular, inspirado no Angular	APIs enterprise, microserviços
Django	Python	"Baterias incluídas", ORM, admin	Aplicações completas, CMS
Flask	Python	Minimalista, flexível	APIs simples, microserviços
Spring	Java	Robusto, enterprise, tipado	Sistemas de grande porte
Laravel	PHP	Elegante, ORM, Blade	Aplicações web completas
Rails	Ruby	"Convention over Configuration", produtivo	Startups, protótipos



Frameworks Front-End

Framework	Linguagem	Características	Ideal para
React	JavaScript/JSX	Biblioteca de UI, virtual DOM, componentes	SPAs, aplicações complexas
Angular	TypeScript	Framework completo, two-way binding	Aplicações enterprise
Vue.js	JavaScript	Progressivo, fácil de aprender	SPAs, projetos de qualquer porte
Svelte	JavaScript	Compilação, sem virtual DOM	Performance, projetos leves
Next.js	React	SSR, SSG, rotas por arquivo	SEO, performance, aplicações completas



Express.js — Exemplo Back-End

```
const express = require('express');
const app = express();

app.use(express.json());

// Rota com middleware
app.get('/api/usuarios', autenticar, async (req, res) => {
  const usuarios = await Usuario.find();
  res.json(usuarios);
});

// Rota com parâmetro
app.get('/api/usuarios/:id', async (req, res) => {
  const usuario = await Usuario.findById(req.params.id);
  if (!usuario) return res.status(404).json({ erro: 'Não encontrado' });
  res.json(usuario);
});

app.listen(3000);
```



React — Exemplo Front-End

```
function ListaUsuarios() {
  const [usuarios, setUsuarios] = useState([]);

  useEffect(() => {
    fetch('/api/usuarios')
      .then(res => res.json())
      .then(data => setUsuarios(data));
  }, []);

  return (
    <ul>
      {usuarios.map(u => (
        <li key={u.id}>{u.nome} – {u.email}</li>
      ))}
    </ul>
  );
}
```



Comparativo: Express vs. Django vs. Laravel

Aspecto	Express	Django	Laravel
Linguagem	JavaScript	Python	PHP
Filosofia	Minimalista	Baterias incluídas	Elegante
ORM	Não (usa plugins)	Sim (integrado)	Eloquent (integrado)
Admin	Não	Sim (integrado)	Não (usa plugins)
Template Engine	Não (usa plugins)	Sim (Django Templates)	Blade
Curva de aprendizagem	Baixa	Média	Média
Flexibilidade	Alta	Média	Média



Síntese da Aula

Conceitos principais:

1. Existem muitos frameworks **Back-End** e **Front-End** — cada um com filosofia diferente;
2. **Express** é minimalista, **Django** é completo, **Laravel** é elegante;
3. **React**, **Angular** e **Vue.js** são os principais frameworks Front-End;
4. A escolha do framework depende do **contexto**, **equipe** e **requisitos**;
5. Não existe "o melhor" framework — existe o **mais adequado** para cada caso.

Pergunta reflexiva:

Se você fosse iniciar um novo projeto hoje, qual framework escolheria e por quê?

Próxima aula: Pull-based × Push-based — Vamos estudar dois paradigmas fundamentais de frameworks.

